SIMATS ENGINEERING

ASSESSMENT TOOL 1: Analytical Problem Solving

COURSE NAME: COMPUTER VISION with OpenCV

COURSE CODE: DSA0204

COURSE OUTCOME COVERED — CO2: *Apply image enhancement and segmentation techniques for extracting meaningful regions from images. (BTL 3)*

Assessment Tool Weightage: 50%

---

1. Grey Level Transformation

(a) Grey level transformation maps every input intensity value r to an output intensity s through a function s = T(r), applied independently to each pixel (point processing). Its purpose is to improve the visual quality or usability of an image — increasing contrast, brightening or darkening selected intensity ranges, or highlighting features of interest — without altering the spatial arrangement of pixels. It forms the foundation for more advanced enhancement operations such as negative, log, and power-law transformations.

Given: *Negative transformation s = 255 − r, applied to r = [50, 100, 150, 200].*

(b) Applying s = 255 − r to each value:

• r = 50 → s = 255 − 50 = 205

• r = 100 → s = 255 − 100 = 155

• r = 150 → s = 255 − 150 = 105

• **r = 200 → s = 255 − 200 = 55**

Transformed values: s = [205, 155, 105, 55]

2. Log Transformation

(a) The log transformation, s = c·log(1+r), expands the range of low (dark) intensity values while compressing the range of high (bright) intensity values, since the logarithmic curve rises steeply for small r and flattens for large r. This is useful for revealing detail in the dark regions of an image (e.g., shadow areas) and for compressing images with a very large dynamic range, such as Fourier spectra, into a displayable range.

Given: *s = c·log(1 + r), c = 10 (natural log), r = [0, 10, 50, 100].*

• r = 0 → s = 10·ln(1) = 10 × 0 = 0.00

• r = 10 → s = 10·ln(11) = 10 × 2.3979 ≈ 23.98

• r = 50 → s = 10·ln(51) = 10 × 3.9318 ≈ 39.32

• r = 100 → s = 10·ln(101) = 10 × 4.6151 ≈ 46.15

(b) Computed values: s ≈ [0.00, 23.98, 39.32, 46.15]

3. Histogram Processing

(a) Histogram equalization redistributes the intensity values of an image so that the resulting histogram is as close to uniform as possible. This spreads out the most frequent intensity values over the available dynamic range, thereby increasing the overall contrast of images whose usable information is concentrated in a narrow band of grey levels.

Given: *3-bit image (L = 8 levels, i.e. 0–7). Histogram: intensity 0 → 2, 1 → 2, 2 → 4, 3 → 8. Total pixels N = 16.*

(b) Step 1 — Probability (PDF): p(0)=2/16=0.125, p(1)=2/16=0.125, p(2)=4/16=0.25, p(3)=8/16=0.5

Step 2 — Cumulative Distribution (CDF): cdf(0)=0.125, cdf(1)=0.25, cdf(2)=0.5, cdf(3)=1.0

Step 3 — Equalized level: s_k = round[(L−1) × cdf(k)] = round[7 × cdf(k)]

• Intensity 0 → round(7 × 0.125) = round(0.875) = 1
• Intensity 1 → round(7 × 0.25) = round(1.75) = 2
• Intensity 2 → round(7 × 0.5) = round(3.5) = 4
• Intensity 3 → round(7 × 1.0) = round(7.0) = 7

Equalized mapping: 0→1, 1→2, 2→4, 3→7

4. Spatial Filtering (Smoothing)

(a) Spatial smoothing filters (e.g., the averaging/mean filter) replace each pixel with the average of the intensities in its neighbourhood. This blurs sharp transitions and reduces random noise, at the cost of some loss of fine detail, and is typically used as a pre-processing step to suppress noise before further operations such as edge detection or segmentation.

Given: *3×3 region: [[10,20,30],[20,30,40],[30,40,50]]. Averaging (mean) filter.*

(b) New center value = sum of all 9 pixels ÷ 9 = (10+20+30+20+30+40+30+40+50) / 9 = 270 / 9 = 30

New center pixel value = 30 (unchanged, since the neighbourhood is already symmetric around the center value of 30)

5. Spatial Filtering (Sharpening)

(a) Sharpening filters enhance fine detail and edges by emphasizing high-frequency components (rapid intensity transitions) of an image. This is commonly done using a second-derivative (Laplacian) operator: the Laplacian response is large near edges and near zero in flat regions, so adding it back to the original image boosts local contrast at edges, making details appear crisper.

Given: *Laplacian mask [[0,−1,0],[−1,4,−1],[0,−1,0]] applied to matrix [[10,10,10],[10,20,10],[10,10,10]].*

(b) Laplacian response at center = (4×20) − (10+10+10+10) = 80 − 40 = 40

Sharpened output = original center + Laplacian response = 20 + 40 = 60

Output at center: Laplacian response = 40; sharpened pixel value = 60

6. Frequency Domain Filtering

(a) In the frequency domain, low frequencies correspond to slowly varying, smooth regions of an image (overall shape/background), while high frequencies correspond to sharp intensity changes (edges, noise, fine texture). A low-pass filter attenuates high-frequency components and keeps the image smooth (blurred), whereas a high-pass filter attenuates low-frequency components and preserves/enhances edges and fine detail.

Given: *F = [100, 80, 40, 20]. Low-pass filter retains only the first two (lowest-frequency) components.*

(b) Retaining the first two components and zeroing the rest:

Filtered output F' = [100, 80, 0, 0]

7. Thresholding

(a) Global thresholding is a segmentation technique that converts a grayscale image into a binary image using a single fixed intensity value T: pixels with intensity greater than or equal to T are classified as foreground (object, usually 1), and pixels below T are classified as background (usually 0). It works best when the object and background occupy clearly separated ranges of intensity.

Given: *Pixel value = 200. Threshold T = 100.*

(b) Since 200 ≥ 100, the pixel is assigned the foreground value.

Thresholded output = 1

8. Edge Detection

(a) Edge detection identifies locations in an image where intensity changes sharply, typically corresponding to object boundaries. It is commonly performed by computing the gradient of the image intensity function; a large gradient magnitude at a pixel indicates a strong edge, while a near-zero gradient indicates a homogeneous (flat) region.

Given: *Sobel horizontal mask [[−1,−2,−1],[0,0,0],[1,2,1]] applied to matrix [[10,10,10],[20,20,20],[30,30,30]].*

(b) Gradient = (−1×10 −2×10 −1×10) + (0+0+0) + (1×30 +2×30 +1×30) = (−40) + 0 + (120) = 80

Gradient at center = 80 (a strong horizontal edge — intensity increases from top row to bottom row)

9. Region-Based Segmentation

(a) Region growing starts from one or more seed pixels and iteratively adds neighbouring pixels to the region if their intensity is sufficiently close (within a specified threshold) to the seed/region intensity. Growth stops when no remaining neighbouring pixel satisfies the similarity criterion, producing a connected homogeneous region.

Given: *Pixel intensities = [48, 50, 52]. Seed = 50, threshold difference = 5.*

(b) Check |pixel − seed| ≤ 5 for each pixel:

- |48 − 50| = 2 ≤ 5 → included
- |50 − 50| = 0 ≤ 5 → included
- |52 − 50| = 2 ≤ 5 → included

All three pixels (48, 50, 52) satisfy the similarity criterion and belong to the grown region.

10. Combined Enhancement & Segmentation

(a) Smoothing is applied before segmentation to suppress noise and small intensity fluctuations that would otherwise be mistaken for genuine object/background boundaries. Segmenting a noisy image directly can create many spurious small regions; smoothing first produces more homogeneous regions so that the subsequent threshold or edge-based segmentation yields cleaner, more meaningful boundaries.

Given: *Image values = [45, 50, 55]. Averaging gives mean = 48 (given). Threshold T = 50.*

(b) Step 1 (averaging): the smoothed value is 48. Step 2 (thresholding): compare 48 with T = 50 → since 48 < 50, the pixel is classified as background (0).

Final segmented output = 0 (background)

11. Power-Law (Gamma) Transformation

(a) Gamma correction, s = c·r^γ, adjusts image brightness non-linearly. It corrects the non-linear intensity response of display/acquisition devices and, more generally, lets us compress or expand a particular range of intensities: γ < 1 expands (brightens) dark regions, while γ > 1 expands (darkens) bright regions and compresses shadows.

(b) With γ = 0.5, the transformation s = r^0.5 (square-root curve) is concave, so it maps low input values to proportionally much higher output values than high input values do. This is justified when an image is too dark and shadow detail needs to be revealed without excessively blowing out the already-bright regions.

Given: *s = c·r^γ, c = 1, γ = 0.5, r = [25, 100, 225].*

(c) Apply step-by-step: s = √r for each value.

- r = 25 → s = √25 = 5
- r = 100 → s = √100 = 10
- r = 225 → s = √225 = 15

(d) Transformed values: s = [5, 10, 15]

(e) Relative to the original spacing (25, 100, 225 — differences of 75 and 125), the transformed values (5, 10, 15 — differences of 5 and 5) are far more evenly spaced and compressed. The darkest input (25) produced a disproportionately larger jump in output than its share of the input range, meaning dark tones gain more relative visibility while bright tones are compressed — consistent with an overall brightening / dynamic-range-compressing effect typical of $\gamma < 1$ correction.

12. Contrast Stretching

(a) An image whose pixel values occupy only a narrow range (here 50–150 out of the possible 0–255) appears flat/low-contrast because the visible tonal range is compressed. Contrast stretching is needed to expand this narrow range across the full display range so that small intensity differences become visually distinguishable.

(b) A linear transformation is justified because it maps the minimum and maximum of the original range directly to the minimum and maximum of the target range while preserving the relative order and proportional spacing of all intermediate values — a simple, distortion-free way to spread out contrast.

Given: *Original range [50, 150] stretched linearly to [0, 255]. $s = (r - 50)/(150 - 50) \times 255 = (r - 50) \times 2.55$*

(c) Apply step-by-step for r = [50, 100, 150]:

- r = 50 → $s = (50-50)/100 \times 255 = 0$
- r = 100 → $s = (100-50)/100 \times 255 = 0.5 \times 255 = 127.5 \approx 128$
- r = 150 → $s = (150-50)/100 \times 255 = 255$

(d) Output values: s ≈ [0, 128, 255]

(e) The stretched output now spans the entire available range (0 to 255) instead of the original narrow 50–150 band. Details that were nearly indistinguishable at close original intensity levels are now separated by much larger output differences, giving a substantial visual improvement in contrast.

13. Histogram Equalization (4-bit Image)

(a) Equalization is needed when pixel intensities are concentrated in a narrow band (as here, all values fall in 0–3 out of the possible 0–15), giving a low-contrast, washed-out appearance. Equalization redistributes these values to better use the full available intensity range.

(b) The cumulative distribution function (CDF) is used because equalization's mapping function is, by definition, a scaled version of the CDF. Using the CDF guarantees a monotonically non-decreasing mapping (preserving intensity order — darker pixels stay darker) while pushing frequently-occurring, tightly clustered levels apart, since the CDF rises fastest where the histogram has the highest concentration of pixels.

Given: *4-bit image (L = 16 levels, 0–15). Histogram: 0→4, 1→6, 2→10, 3→6. N = 26.*

(c) Cumulative frequency: cdf(0) = 4, cdf(1) = 4+6 = 10, cdf(2) = 10+10 = 20, cdf(3) = 20+6 = 26

(d) New level = round[(L−1) × cdf(k)/N] = round[15 × cdf(k)/26]:

- Level 0 → round(15×4/26) = round(2.31) = 2
- Level 1 → round(15×10/26) = round(5.77) = 6
- Level 2 → round(15×20/26) = round(11.54) = 12
- Level 3 → round(15×26/26) = round(15.00) = 15

Equalized mapping: 0→2, 1→6, 2→12, 3→15

(e) The four original levels, which were all crowded into the narrow range 0–3, are now spread across nearly the entire 0–15 range (2, 6, 12, 15). This wider spacing between output levels means the same four intensity groups now produce much greater visual contrast in the displayed image.

14. Median Filtering

(a) Salt-and-pepper (impulse) noise randomly replaces isolated pixels with extreme high or low intensity values, while the rest of the image is unaffected. In the given neighbourhood, the value 50 (true signal) is surrounded by alternating extreme values of 10 (pepper-like) and 200 (salt-like), a classic impulse-noise pattern.

(b) The median filter is justified because it is a rank-order (non-linear) filter: it replaces each pixel with the median of its neighbourhood rather than a weighted sum. Since impulse noise pixels are statistical outliers, they get pushed to the extremes of the sorted list and are naturally excluded from the median, whereas a linear (mean) filter would still let them skew the result. Median filtering therefore removes impulse noise while preserving edges much better than averaging.

Given: *3×3 neighbourhood: [[10,200,10],[200,50,200],[10,200,10]].*

(c) Sort all 9 values: 10, 10, 10, 10, 50, 200, 200, 200, 200 (four 10's, one 50, four 200's).

(d) The median is the 5th value in the sorted list of 9 → median = 50.

Center value after median filtering = 50

(e) The filtered center value (50) matches the true underlying signal exactly, while the surrounding impulse noise values (10 and 200) were completely discarded from the computation. This confirms the median filter's strong effectiveness at removing salt-and-pepper noise without blurring the genuine pixel value.

15. Gaussian Smoothing

(a) The objective of Gaussian smoothing is to reduce noise by blurring the image using weights derived from a Gaussian (bell-curve) distribution, so that the influence of each neighbouring pixel decreases smoothly with its distance from the center — giving a more natural, less blocky blur than simple averaging.

(b) A Gaussian filter is preferred over a plain averaging filter because it weights the center and near pixels more heavily than distant ones, better matching the way real optical/sensor blur behaves. It is also separable and rotationally symmetric (isotropic), and it produces smoother transitions with less ringing and better edge preservation than a uniform box filter of the same size.

Given: *Gaussian mask (unnormalized) [[1,2,1],[2,4,2],[1,2,1]], sum = 16, applied to matrix [[10,20,10],[20,40,20],[10,20,10]].*

(c) Weighted sum = (1×10+2×20+1×10) + (2×20+4×40+2×20) + (1×10+2×20+1×10) = (10+40+10) + (40+160+40) + (10+40+10) = 60 + 240 + 60 = 360

(d) Center pixel value = 360 / 16 = 22.5

(e) The smoothed center value (22.5) is pulled from the original 40 toward the average of its (lower-valued) neighbours, reducing the local peak while retaining the overall trend of the region. This illustrates how Gaussian smoothing suppresses local noise/spikes while keeping the broader intensity structure largely intact.

16. High-Pass Filtering

(a) High-pass filtering is needed when an image (or a region of interest within it) appears blurred or lacks crisp detail; by passing through high-frequency components (rapid intensity changes) and suppressing low-frequency (smooth) content, it emphasizes edges and fine detail, sharpening the visual appearance.

(b) The given mask, with a strong positive center weight (+8) and negative surrounding weights (−1 each, summing to −8 so the kernel sums to 0), computes the difference between the center pixel and the average of its neighbours — precisely the high-frequency (edge) content at that location — making it a standard discrete high-pass/Laplacian-type operator.

Note: The question did not supply an accompanying pixel matrix for this mask, so the 3×3 block used in Question 5 ([[10,10,10],[10,20,10],[10,10,10]]) is reused here for a concrete, worked demonstration.

Given: *Mask [[−1,−1,−1],[−1,8,−1],[−1,−1,−1]] applied to matrix [[10,10,10],[10,20,10],[10,10,10]].*

(c) Output = 8×20 − (10+10+10+10+10+10+10+10) = 160 − 80 = 80

(d) Center value = 80

(e) The large positive response (80) at the center — much greater than the original pixel value (20) — indicates a strong local intensity peak relative to its surroundings. Applying this filter over the whole image accentuates such peaks and edges, producing a visibly sharper output.

17. Frequency Domain High-Pass

(a) High-frequency components in the frequency domain correspond to rapid spatial intensity transitions — edges, fine texture, and noise. They carry the detail information that defines object boundaries and shape, whereas low frequencies mainly capture the smooth overall brightness/shading of the image.

(b) High-pass filtering is justified when the goal is to enhance or extract edges and detail: by suppressing the low-frequency (bulk/background) content and retaining only high-frequency components, the filtered result highlights exactly the regions of rapid change.

Given: *F = [10, 20, 80, 100]. High-pass filter removes the low-frequency components.*

(c) Remove (zero out) the first two, lowest-frequency components and retain the last two, highest-frequency components.

(d) Output F' = [0, 0, 80, 100]

(e) The retained components (80, 100) represent the fine detail/edge information of the image, while the suppressed components (10, 20) — which represented the smooth background — are eliminated. The result emphasizes sharp transitions and detail over broad, gradual shading.

18. Adaptive Thresholding

(a) A single global threshold can fail when illumination varies across the image (e.g., shadows or uneven lighting), because a pixel that is genuinely part of the foreground in a dim region might have the same intensity as a background pixel in a brightly lit region — a global threshold cannot distinguish the two correctly.

(b) Adaptive thresholding computes the threshold locally (e.g., from the local mean or a weighted local statistic in each pixel's neighbourhood) instead of using one fixed value for the entire image. This allows the decision boundary to adjust to local lighting conditions, giving more accurate segmentation under non-uniform illumination.

Given: *Pixel values = [60, 80, 120, 140]. Local threshold = local mean = 100.*

(c) Compare each pixel with the local mean of 100: value $\geq 100 \rightarrow 1$ (foreground); value $< 100 \rightarrow 0$ (background).

- $60 < 100 \rightarrow 0$
- $80 < 100 \rightarrow 0$
- $120 \geq 100 \rightarrow 1$
- $140 \geq 100 \rightarrow 1$

(d) Binary output = [0, 0, 1, 1]

(e) Pixels below the local mean (60, 80) are classified as background, and pixels above it (120, 140) as foreground/object — correctly separating the two groups relative to their local intensity context rather than a single fixed global cut-off.

19. Edge Detection (Vertical Sobel)

(a) This operator detects vertical edges — i.e., locations where intensity changes rapidly from left to right — by approximating the horizontal gradient of the image while smoothing along the vertical direction.

(b) The vertical Sobel mask is chosen because its extra weighting (±2 in the middle row) makes it more robust to noise than a simple 1-pixel-wide gradient operator, while still responding strongly and specifically to horizontal intensity changes (vertical edges).

Given: *Mask [[−1,0,1],[−2,0,2],[−1,0,1]] applied to matrix [[10,20,30],[10,20,30],[10,20,30]].*

(c) Row-wise: Row1 = (−1×10)+(0×20)+(1×30) = 20; Row2 = (−2×10)+(0×20)+(2×30) = 40; Row3 = (−1×10)+(0×20)+(1×30) = 20. Total = 20+40+20 = 80

(d) Center value (gradient) = 80

(e) The large positive gradient (80) confirms a strong vertical edge at this location, with intensity increasing from left (10) to right (30) — consistent with the uniform left-to-right ramp pattern in the given matrix.

20. Prewitt Edge Detection

(a) The Prewitt operator detects edges by approximating the intensity gradient using simple, equally-weighted differences between neighbouring pixel columns (or rows), highlighting locations of rapid intensity change in a chosen direction.

(b) Compared with Sobel, the Prewitt mask uses uniform weights (1, 1, 1) rather than Sobel's weighted (1, 2, 1) row, making it computationally simpler but slightly less effective at suppressing noise; Sobel's extra central weighting gives a smoother, more noise-robust gradient estimate, which is why Sobel is generally preferred in noisy conditions while Prewitt is preferred for its simplicity.

Note: As with Question 19, the same matrix [[10,20,30],[10,20,30],[10,20,30]] is used here so the Prewitt result can be directly compared with the Sobel result above.

Given: *Mask [[−1,0,1],[−1,0,1],[−1,0,1]] applied to matrix [[10,20,30],[10,20,30],[10,20,30]].*

(c) Row-wise: Row1 = (−1×10)+(0×20)+(1×30) = 20; Row2 = 20; Row3 = 20. Total = 20+20+20 = 60

(d) Center value (gradient) = 60

(e) The Prewitt output (60) is somewhat lower than the Sobel output (80) on the same data, because Prewitt weights all three rows equally while Sobel emphasizes the middle row — showing numerically why Sobel gives a stronger, noise-weighted response while Prewitt gives a simpler, unweighted one.

21. Region Splitting

(a) The pixel set contains two distinct intensity clusters — around 40–45 and around 100–105 — which likely correspond to two different regions or objects in the image that need to be separated from each other.

(b) Region splitting is justified because it recursively divides an image (or pixel set) into sub-regions based on a homogeneity test, continuing to split until every resulting region is internally uniform (within a chosen tolerance) — an appropriate approach when the data clearly falls into separable intensity clusters, as here.

Given: *Pixel set = [40, 45, 100, 105]. Split threshold = 80.*

(c) Classify each pixel relative to the threshold of 80: value < 80 → Region A; value ≥ 80 → Region B.

(d) Region A = {40, 45}; Region B = {100, 105}

(e) The threshold of 80 falls cleanly between the two clusters (45 and 100), so the split produces two homogeneous regions that correctly separate the two underlying objects/backgrounds with no ambiguous pixels.

22. Region Merging

(a) Adjacent regions whose intensities (or means) are close to each other likely belong to the same underlying object; region merging combines such regions to correct the over-segmentation that region-splitting or seed-based growing methods can produce.

(b) Merging based on the intensity difference between adjacent regions' boundary/mean values is justified because it directly tests the homogeneity assumption between neighbours — if the difference is small, treating them as separate regions would be an artificial, unnecessary fragmentation of what is really one region.

Given: *Region 1 = [45, 50], Region 2 = [52, 55]. Merge threshold = 5.*

(c) Compare the closest boundary values of the two regions: last value of Region 1 (50) and first value of Region 2 (52): difference = |52 − 50| = 2, which is ≤ 5, so the regions qualify for merging.

(d) Merged region = {45, 50, 52, 55}, with combined mean = (45+50+52+55)/4 = 202/4 = 50.5

(e) Since the boundary intensity difference (2) is well within the merge threshold (5), the two regions are correctly combined into a single, larger homogeneous region representing one object rather than being left as two artificially separate fragments.

23. Edge Linking

(a) Raw edge detection often produces broken, discontinuous sequences of edge pixels — due to noise or locally weak gradients — rather than one continuous boundary. In the given sequence [1, 0, 1, 0, 1], the 0's represent such gaps interrupting an otherwise continuous edge.

(b) Edge linking is justified because it analyses nearby edge pixels (based on proximity and similarity of gradient direction/magnitude, e.g., via local search or the Hough transform) and connects them across small gaps, reconstructing a continuous boundary from fragmented detections.

Given: *Detected edge sequence = [1, 0, 1, 0, 1].*

(c) Each gap (0) is bounded on both sides by an edge pixel (1), so under a small-gap linking rule these isolated gaps are filled in and marked as part of the edge.

(d) Linked result = [1, 1, 1, 1, 1]

(e) After linking, the sequence becomes a single unbroken run of edge pixels, correcting the discontinuities that were most likely caused by noise or a locally weak gradient rather than an actual gap in the true object boundary.

24. Low-Pass Filtering (Frequency)

(a) To smooth an image and suppress noise, the high-frequency components — which represent rapid, often noise-driven intensity fluctuations — should be attenuated, while the low-frequency components, which represent the image's overall smooth structure, should be preserved.

(b) A low-pass filter is the correct choice here because it passes through exactly the low-frequency content that carries the meaningful smooth structure of the image while discarding the high-frequency content, most of which corresponds to noise rather than genuine fine detail in this context.

Given: *F = [200, 150, 50, 20]. Low-pass filter retains the first two (lowest-frequency) components.*

(c) Retain the first two components; zero out the remaining, higher-frequency components.

(d) Filtered output F' = [200, 150, 0, 0]

(e) The result preserves the dominant, smooth structure of the signal/image (200, 150) while removing the smaller, high-frequency components (50, 20) that most likely correspond to noise — yielding a smoother, denoised version at the cost of some very fine detail.

25. Laplacian Sharpening (Variant)

(a) Sharpening is needed whenever an image's edges and fine detail appear soft or blurred; boosting the high-frequency (edge) content relative to the smooth background restores perceived crispness.

(b) This mask (center weight +5, four immediate neighbours −1 each, corner neighbours 0) is a composite Laplacian-sharpening kernel whose coefficients sum to 1. Unlike a pure Laplacian (which sums to 0 and only extracts edge information), this kernel directly outputs the sharpened image in a single pass — equivalent to computing the Laplacian and adding it back to the original pixel in one step.

Note: No specific pixel matrix was supplied with this question, so — since this mask is a variant of the Laplacian mask in Question 5 — the same matrix [[10,10,10],[10,20,10],[10,10,10]] is reused for a concrete, comparable computation.

Given: *Mask [[0,−1,0],[−1,5,−1],[0,−1,0]] applied to matrix [[10,10,10],[10,20,10],[10,10,10]].*

(c) Output = (0×10) + (−1×10) + (0×10) + (−1×10) + (5×20) + (−1×10) + (0×10) + (−1×10) + (0×10) = −10 + (−10+100−10) + −10 = −10 + 80 − 10 = 60

(d) Center value = 60

(e) The output (60) is higher than the original center pixel (20), directly reflecting the sharpening effect — the local intensity peak has been emphasized relative to its uniformly darker (value 10) surroundings, in a single-pass computation equivalent to the two-step process used in Question 5.

26. Histogram Matching

(a) Histogram matching (specification) is needed when a specific target appearance (a desired histogram shape) is required — for example, matching the tone of several images taken under different lighting conditions to a common reference — which simple equalization (which only aims for a uniform histogram) cannot guarantee.

(b) The method works by first equalizing both the input image's histogram and the desired reference/target histogram to obtain their respective cumulative distribution functions (CDFs). Each input intensity is then mapped to the equalized value via the input CDF, and that equalized value is mapped backward through the inverse of the reference CDF to find the corresponding output intensity — ensuring the output image's histogram matches the reference as closely as possible.

Note: The question specifies input values to map [10, 50, 100] but does not supply a target/reference histogram, without which an exact numeric mapping cannot be computed. The general procedure is illustrated conceptually below, with the input values treated as approximately preserving their relative rank order under the mapping — the typical qualitative behaviour of histogram matching.

(c) Conceptually: G(r) = CDF_input(r) is computed for each input value, and each is then mapped to the z that satisfies CDF_reference(z) = G(r).

(d) Without a specified reference histogram, only the relative behaviour can be stated: since histogram matching preserves intensity order (it is a monotonically non-decreasing mapping), the mapped outputs for r = [10, 50, 100] would remain in the same increasing order, e.g. z < z < z , with their exact spacing determined entirely by the shape of the reference histogram.

(e) The key result is qualitative rather than numeric here: histogram matching reshapes the input's tonal distribution to resemble a chosen reference distribution rather than forcing a uniform one, which is useful for achieving a consistent, targeted look across images — the specific numeric output values [10, 50, 100] would map to depend entirely on which reference histogram is supplied.

27. Noise Reduction Comparison

(a) The noise here is salt-and-pepper (impulse) noise: isolated pixels are randomly replaced by extreme high (200) or low (10) values while the true underlying signal (50) remains at the center — this differs from Gaussian noise, which perturbs every pixel by a small random amount rather than a few pixels by a large amount.

(b) Averaging (mean) filtering computes a linear combination of all neighbourhood values, so extreme outlier values still contribute proportionally to the result, pulling the output away from the true signal. Median filtering, being rank-based, discards outliers entirely (they simply end up at the ends of the sorted list) and is therefore far more effective against impulse noise, while also preserving edges better than averaging.

Given: *Same 3×3 neighbourhood as Question 14: [[10,200,10],[200,50,200],[10,200,10]].*

(c) Averaging: sum = 10+200+10+200+50+200+10+200+10 = 890; mean = 890/9 ≈ 98.89. Median: sorted values are 10,10,10,10,50,200,200,200,200; median (5th value) = 50.

(d) Average output ≈ 98.89; Median output = 50

(e) The median output (50) is identical to the true underlying pixel value, whereas the average output (≈98.89) is heavily distorted by the surrounding impulse-noise pixels — a clear numeric demonstration of the median filter's superiority for salt-and-pepper noise.

28. Combined Filtering

(a) Smoothing followed by sharpening is used because raw sharpening (a high-pass/derivative operation) amplifies high-frequency content indiscriminately — including noise. Smoothing first suppresses the noise, and sharpening is then applied to the cleaner image so that only genuine edges, not noise, are emphasized.

(b) The sequence (smooth, then sharpen) is justified rather than the reverse because sharpening a noisy image first would amplify the noise itself into large spurious high-frequency responses; a subsequent smoothing pass would then blur these amplified noise artifacts together with the genuine edges, degrading the result. Smoothing first avoids this compounding of noise and detail.

Given: *Reusing the Gaussian mask/matrix from Question 15 conceptually to illustrate the two-stage pipeline.*

(c) Stage 1 (smoothing): from Question 15, the Gaussian-smoothed center value is 22.5 (reduced from the original 40, with noise/local variation suppressed). Stage 2 (sharpening): a Laplacian-type sharpening mask (as in Question 5) would then be applied to this smoothed neighbourhood to restore edge contrast.

(d) A full numeric result for stage 2 requires the entire smoothed 3×3 neighbourhood (not just the single center value produced in Question 15), which is not available here; qualitatively, the final output would lie above the purely smoothed value of 22.5 (as sharpening restores some contrast) but below the original unsmoothed peak of 40 (since noise-driven excess has already been removed).

(e) This two-stage pipeline achieves both noise suppression and edge enhancement — an improvement over sharpening alone, which would have re-amplified whatever noise was present in the original data.

29. Multi-Threshold Segmentation

(a) Unlike simple global thresholding, which only produces a binary (two-class) result, multi-level thresholding partitions pixel intensities into three or more classes — appropriate when an image contains more than one distinct object or region against the background, each occupying a different intensity band.

(b) The thresholds 50 and 100 are justified because they fall in the gaps between the clusters of the given pixel values (20 | 60 | 120, 180), cleanly separating them into three natural intensity bands without splitting any single cluster.

Given: *Pixels = [20, 60, 120, 180]. Thresholds at 50 and 100 define three classes: Class 1 (<50), Class 2 (50–100), Class 3 (>100).*

(c) Classify each pixel against the two thresholds.

(d) 20 → Class 1; 60 → Class 2; 120 → Class 3; 180 → Class 3

(e) The two thresholds successfully separate the four pixel values into three meaningful intensity groups (dark/background, mid-tone, bright/object region), which a single global threshold could not have distinguished, since it would have only produced two classes.

30. Boundary Detection

(a) Boundary (contour) extraction identifies the closed outline separating an object region from the background. It is a key step for shape-based analysis, since features such as perimeter, area, and shape descriptors are computed from the boundary rather than from the full region.

(b) Combining edge detection with edge linking is justified because gradient-based edge detectors (e.g., Sobel/Prewitt) typically produce fragmented edge points rather than a single closed contour; edge linking is then needed to connect these fragments — following continuity and direction cues — into one continuous, closed boundary.

Given: *Conceptual pipeline building on Questions 8/19 (edge detection) and Question 23 (edge linking).*

(c) Step 1: apply a gradient operator (e.g., Sobel) to obtain raw candidate edge points, which — as seen in Question 23 — may be fragmented (e.g., [1,0,1,0,1]). Step 2: apply edge linking to bridge the small gaps between these fragments.

(d) Resulting boundary (analogous to Question 23's linked output): a fully connected sequence, e.g. [1,1,1,1,1] — one continuous, closed contour.

(e) The final linked boundary accurately traces the object's outline as a single continuous contour, enabling reliable downstream shape analysis (perimeter, area, classification) that fragmented, disconnected edge points alone could not support.

Code of Conduct:

I V M Sai Bhragav certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: V M Sai Bhargav

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation